

PREFACE FOR FACULTY

The **Overlap-Save (OLS)** method (also called Overlap-Discard) is an alternative block-filtering method where input blocks overlap by $M - 1$ samples. Rather than adding output tails as in OLA, the corrupted aliased samples are simply discarded from the circular convolution output.

Pedagogical Strategy:

1. Formulate input block partitioning with $M - 1$ sample overlap: $x_m[n] = x[mL + n - (M - 1)]$.
 2. Explain why the first $M - 1$ points of circular convolution represent wrap-around aliasing.
 3. Demonstrate that the remaining L points exactly equal the linear convolution.
 4. Compare Overlap-Add and Overlap-Save:
 - OLA: Disjoint inputs $\rightarrow$ Overlapping output additions.
 - OLS: Overlapping inputs $\rightarrow$ Discard aliased output prefix; direct concatenation.
 5. Explain why OLS is preferred in SIMD/GPU architectures because it requires zero output additions (pure memory copy).
-

1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Formulate** overlapping input blocks for the Overlap-Save algorithm.
 2. **Execute** numerical filtering via Overlap-Save and identify aliased regions.
 3. **Compare** OLA and OLS in terms of memory access, register operations, and arithmetic complexity.
 4. **Select** optimal block sizes for real-time DSP implementation.
-

2. MATHEMATICAL FOUNDATIONS

2.1 Overlap-Save Block Formulation

Let FIR filter $h[n]$ have length M . Choose block length L such that total FFT length is $N = L + M - 1$. Construct input blocks $x_m[n]$ of length N by prepending the last $M - 1$ points from block $m - 1$:

$$x_m[n] = x[mL + n - (M - 1)], \quad 0 \leq n \leq N - 1$$

For the first block ($m = 0$), prepend $M - 1$ zeros:

$$x_0[n] = \underbrace{0, 0, \dots, 0}_{M-1 \text{ zeros}}, x[0], x[1], \dots, x[L - 1]$$

2.2 Circular Convolution & Discard Mechanism

Zero-pad $h[n]$ to length N . Compute the N -point circular convolution:

$$\tilde{y}_m[n] = x_m[n] \otimes_N h[n] = \text{IDFT}_N \text{DFT}_N x_m \cdot \text{DFT}_N h$$

The output contains:

- Samples $n = 0, 1, \dots, M - 2$: **Corrupted by circular wrap-around aliasing** $\implies$ **DISCARD**.
- Samples $n = M - 1, M, \dots, N - 1$: **Valid linear convolution samples** $\implies$ **SAVE**.

2.3 Synthesis of Output

The total linear convolution $y[n]$ is formed by direct concatenation of the saved portions:

$$y[mL + r] = \tilde{y}_m[r + M - 1], \quad 0 \leq r \leq L - 1$$

3. WORKED NUMERICAL EXAMPLES

Example 14.1: Complete Overlap-Save Filtering

Problem: Filter $x[n] = \underset{\uparrow}{1}, 2, -1, 2, 3, -2, 0, 1, 2, 1$ with $h[n] = \underset{\uparrow}{1}, 2, 1$ ($M = 3$) using Overlap-Save with $N = 6$ ($L = 4$).

Solution:

- $M = 3 \implies M - 1 = 2$ overlap samples. $N = 6 \implies L = N - M + 1 = 4$.
- **Construct Blocks of Length $N = 6$:**
 - $x_0[n] = 0, 0, 1, 2, -1, 2$ (Prepended 2 zeros)
 - $x_1[n] = -1, 2, 3, -2, 0, 1$ (Overlap from last 2 of block 0: $-1, 2$)
 - $x_2[n] = 0, 1, 2, 1, 0, 0$ (Overlap: $0, 1$; plus zero padding at end)
- **Compute 6-Point Circular Convolution** ($\tilde{y}_m = x_m \otimes_6 1, 2, 1, 0, 0, 0$):
 - $\tilde{y}_0 = \{ \underbrace{3, 2}_{\text{Discard}}, \mathbf{1, 4, 4, 1} \}$
 - $\tilde{y}_1 = \{ \underbrace{2, 1}_{\text{Discard}}, \mathbf{6, 6, -1, 0} \}$
 - $\tilde{y}_2 = \{ \underbrace{2, 1}_{\text{Discard}}, \mathbf{4, 6, 4, 1} \}$
- **Concatenate Saved Parts:**

$$y[n] = \mathbf{1, 4, 4, 1, 6, 6, -1, 0, 4, 6, 4, 1}$$

Result:

$$y[n] = \underset{\uparrow}{1}, 4, 4, 1, 6, 6, -1, 0, 4, 6, 4, 1$$

Identical to the Overlap-Add result.

4. UNIVERSITY EXAMINATION QUESTIONS & MARKING RUBRIC

Question 1 (15 Marks)

(a) Compare the Overlap-Add and Overlap-Save methods in detail. Under what conditions is Overlap-Save preferred? (7 Marks) **(b)** Filter $x[n] = \underset{\uparrow}{2}, -1, 3, 1, 2, 0, 1, 4$ with $h[n] = \underset{\uparrow}{1}, -1$ using the Overlap-Save method with $N = 4$ ($L = 3$). (8 Marks)

Model Answer & Step-by-Step Marking Rubric:

- **Part (a):**
 - Detailed structural comparison covering input buffering, convolution, and output synthesis (4 Marks)
 - Explanation: OLS avoids output additions, making it ideal for DMA streaming and parallel SIMD/GPU memory copy pipelines (3 Marks)
- **Part (b):**
 - $M = 2 \implies M - 1 = 1$ overlap sample. $N = 4, L = 3$.
 - Block partitioning: $x_0 = 0, 2, -1, 3$ $x_1 = 3, 1, 2, 0$ $x_2 = 0, 1, 4, 0$ (2 Marks)
 - Circular convolution with $h = 1, -1, 0, 0$: $\tilde{y}_0 = 0, 2, -1, 3 \otimes_4 1, -1, 0, 0 = -3, \mathbf{2}, -\mathbf{3}, \mathbf{4}$ (Discard index 0) $\tilde{y}_1 = 3, 1, 2, 0 \otimes_4 1, -1, 0, 0 = 3, -\mathbf{2}, \mathbf{1}, -\mathbf{2}$ (Discard index 0) $\tilde{y}_2 = 0, 1, 4, 0 \otimes_4 1, -1, 0, 0 = 0, \mathbf{1}, \mathbf{3}, -\mathbf{4}$ (Discard index 0) (4 Marks)
 - Concatenation of saved parts: $y[n] = 2, -3, 4, -2, 1, -2, 1, 3, -4$ (2 Marks)

5. PYTHON VERIFICATION SCRIPT

```
import numpy as np

x = np.array([2, -1, 3, 1, 2, 0, 1, 4])
h = np.array([1, -1])
y = np.convolve(x, h)
print("Direct Linear Convolution:", y)
```